INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

SMALL INTRO TO GIT

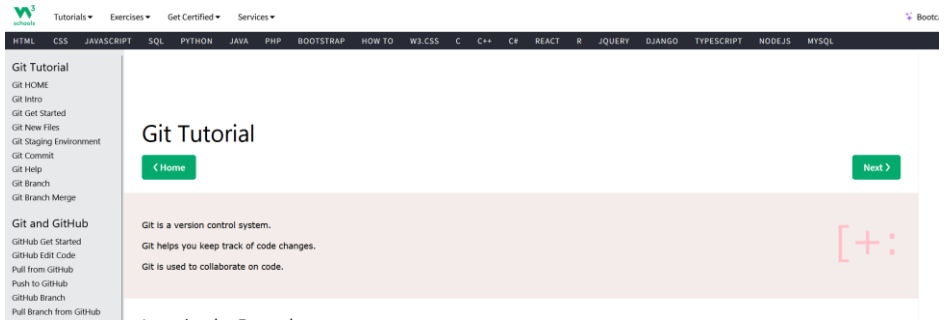
A version control system

S. Delprat

Sources

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

This document is deeply inspired by this tutorial :
https://www.w3schools.com/git



Other tutorials:

- https://www.atlassian.com/git/tutorials
- https://www.w3docs.com/learn-git/
- https://buzut.net/cours/versioning-avec-git/historique

Main Principles

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

GIT allow to :

- Track changes in files
- Revert changes in file
- Find differences between versions
- Manage different versions of code
- Much more…

GIT can be coupled with GitHub, GitLab

- Backup files & their history
- Allows multiple users to collaborate
- etc

The important thing is that all the changes can be tracked and reverted if needed, even when multiple persons works on the same project. Git also helps to ensure that the main code is working and validated.

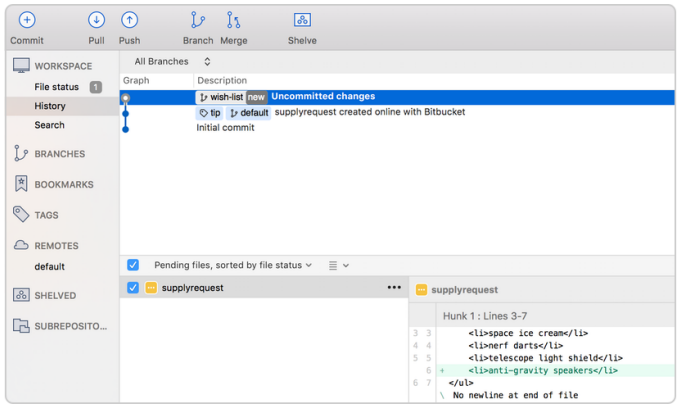
Graphical User Interfaces ?

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

GIT is a command line tool.
You're not a geek? A few Graphical User Interfaces are available.
=> They can be helpful to visualize the different code merges & branches

Sourcetree

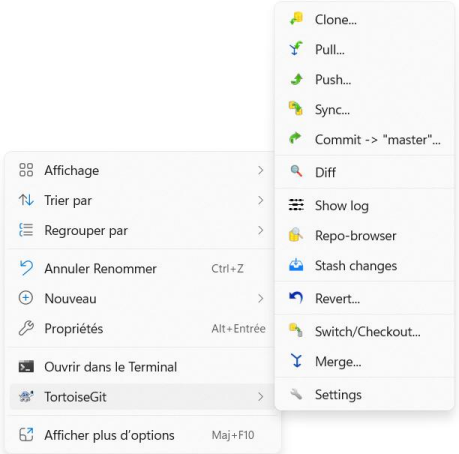
https://www.sourcetreeapp.com/



https://tortoisegit.org/

TortoiseGit
Windows Shell Interface to Git

This is a windows explorer extension allowing to perform most of GIT command by click



Git configuration

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Once Git is installed, its time to check if it’s working

```
git --version
```

In order to track who made changes to the code, it is necessary to identify yourself:

```
git config --global user.name “Alexandre Dupont“
git config --global user.email “alexandre.dupont@nowhere.com"
```

git init --initial-branch=main

Create a first repository

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- Create an empty folder and initialize a new repository

```
git init
```

Nom	Modifié le
.git	06/07/2023 09:16

Git automatically creates a hidden folder “.git” to manage this repository

The default name of the branch is “master”. You may prefer this name to “main”

```
git init --initial-branch=main
```

- Lets add some code and save file (in the repository folder) as `exeDemo1.m`

```
clearvars
close all
clc

% This is a first file into the repository
fprintf('My first file\n');
```

Create a first repository

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- Get the status of the repository

```
git status
```

As Git monitors the current folder, it is aware that a new file is located in the folder ,
It also inform us that this file is not added to the repository

```
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        exeDemo1.m

nothing added to commit but untracked files present (use "git add" to track)
```

Files can have on of this status:

- Untracked : the file are located in the repository, but not added to the repository (not tracked by Git)
- Tracked : the file is in the repository and it is tracked by Git

File status

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

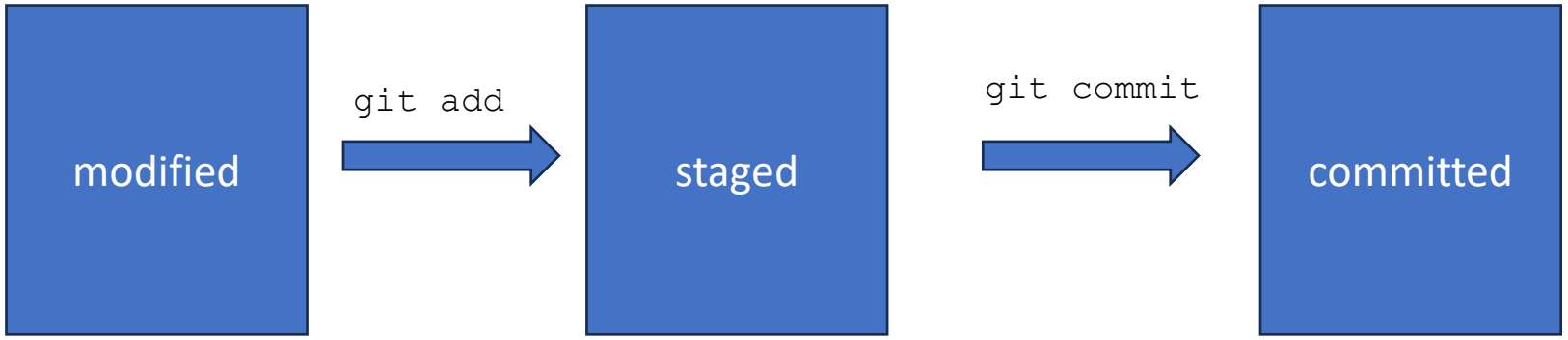
A file can be in 3 main states:

- Modified : you are working on a file and it is modified
- Staged : you put the current content of a file in a list of files that will be later saved

  Typically used when 1 file is "ready" but the project overall need more work.
  Staging is a kind of "intermediate" save.

- Committed : This means that the files content has been definitively saved in a snapshot

  Typically, a commit is done when some achievement have been reached.



Stage files

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- Get the status of the repository

```
git add exeDemo1.m
```

Now with `git status`, we can see that a exeDemo1

```
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   exeDemo1.m
```

There is a new file which is “monitored” by git.

Add one file

```
git add filename
```

Add one folder

```
git add folder
```

Ajouter tout

```
git add -all
```

```
git add -A
```

Commit

Now all the file is added, we can do the first commit

```
git commit -m "Message : explanation of this commit"
```

```
PS D:\seb\Cours\INSA\Git\demo> git commit -m "initial commit"
[master (root-commit) 4166dc1] initial commit
 1 file changed, 7 insertions(+)
 create mode 100644 exeDemo1.m
```

Now our repository history is filled with all the needed data.

More files…

Now let's add 2 more code file + 1 image into the folder

`displayImage.m`

```
function displayImage(fileName)
[img,map]=imread(fileName);
figure;imshow(img,map);
end
```

`exeDemo2.m`

```
clearvars
close all
clc
displayImage('gitLogo.png')
```

`gitLogo.png`

Let's check the repo status using the `short` option

```
PS D:\seb\Cours\INSA\Git\demo> git status --short
?? displayImage.m
?? exeDemo2.m
?? gitLogo.png
```

Flags:

- ?? : untracked file
- A : Files added to stage
- M : Modified files
- D : Deleted files

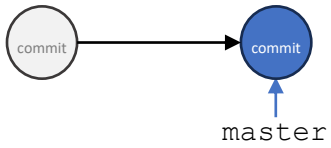
More files...

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Finally, stage the files and commit

```
PS D:\seb\Cours\INSA\Git\demo> git add -A
PS D:\seb\Cours\INSA\Git\demo> git commit -m "Added three new files"
[master 25230db] Added three new files
 3 files changed, 9 insertions(+)
 create mode 100644 displayImage.m
 create mode 100644 exeDemo2.m
 create mode 100644 gitLogo.png
```

Now our repository history contains the information of the initial commit and the new commit



As a result, git will allow us to cancel any changes and much more!

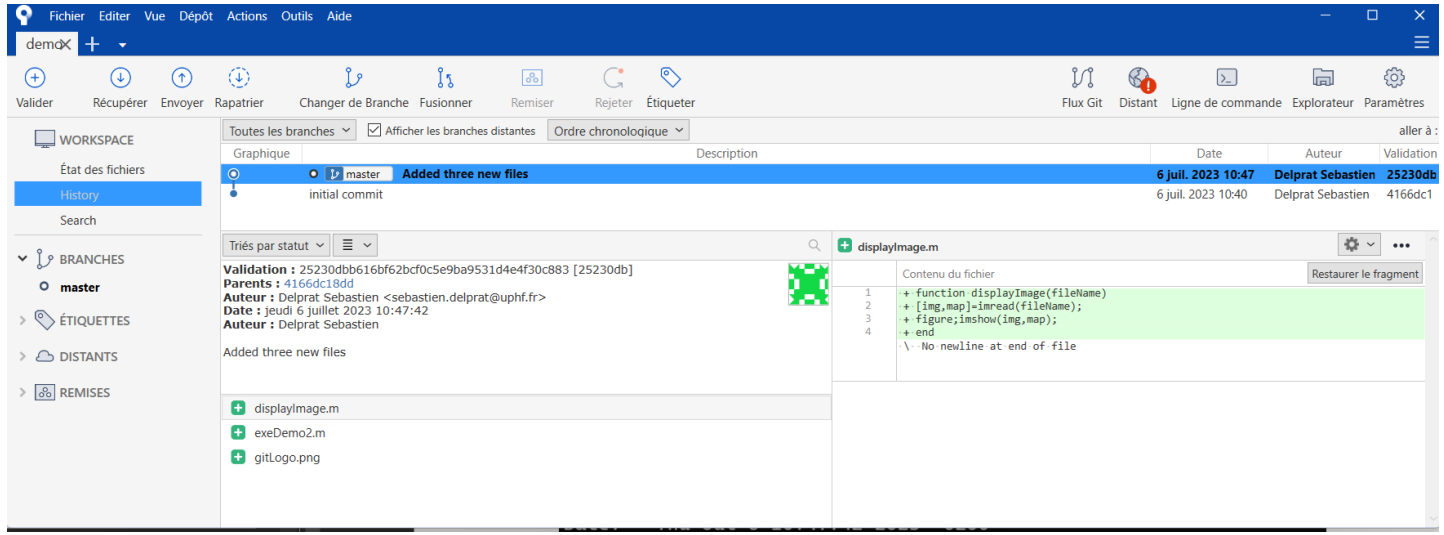
Git log

We can track the history of the repository

```
git add exeDemo1.m
```

```
PS D:\seb\Cours\INSA\Git\demo> git log
commit 25230dbb616bf62bcf0c5e9ba9531d4e4f30c883 (HEAD -> master)
Author: Delprat Sebastien <sebastien.delprat@uphf.fr>
Date:   Thu Jul 6 10:47:42 2023 +0200

    Added three new files

commit 4166dc18dd38870162eea425ca86721e9a38add2
Author: Delprat Sebastien <sebastien.delprat@uphf.fr>
Date:   Thu Jul 6 10:40:31 2023 +0200
```

As it is not really user friendly, we may want to use a GUI such as source tree:



Branches

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

In order to modify the repository code, it is advised to create a new branch : a copy of the original repository

Git will later allow to securely apply modifications done on the new branch to the original repository.

```
git status
```

```
PS D:\seb\Cours\INSA\Git\demo> git status
On branch master
nothing to commit, working tree clean
```

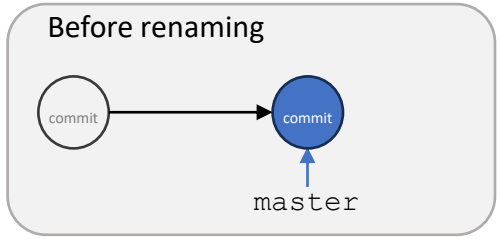
We are working on the “master” branch

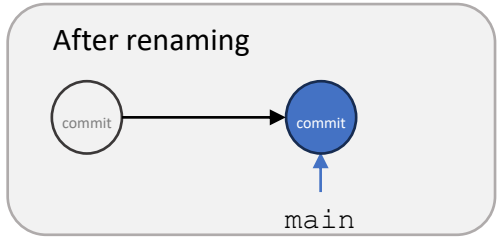
NB : gitlab now use “main” as default name

If you want to rename a branch:

```
git branch -m newBranchName
```

```
PS D:\seb\Cours\INSA\Git\demo> git branch -m main
PS D:\seb\Cours\INSA\Git\demo> git status
On branch main
nothing to commit, working tree clean
```

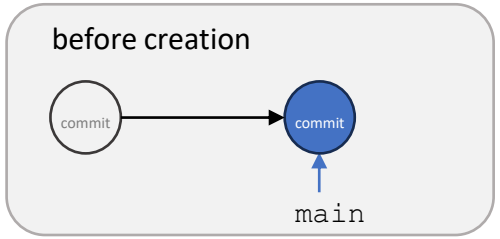




Branches

- Let’s create a new branch

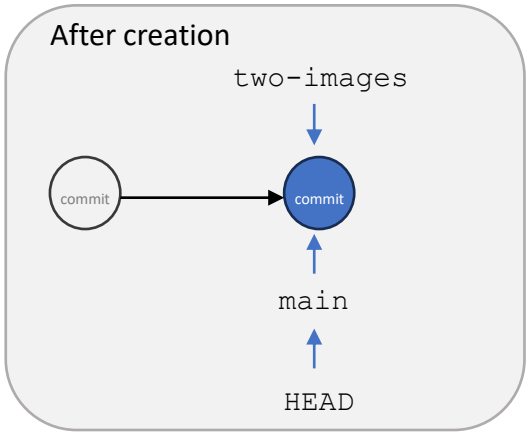
```
git branch two-images
```



Internally, a branch is simply a "pointer" to one of the commit.
Each time you do a commit, the "pointer" is moved to the new commit.

The active branch is managed through a pointer called "`HEAD`" that can be moved using the checkout command.

As we didn't move yet to the new branch, the `HEAD` still points to the `main` branch.



Branches

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- We can list the branch of the repository

```
git branch
```

Active branch

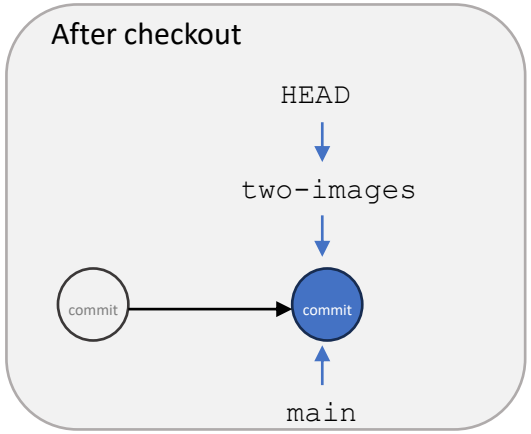
```
PS D:\seb\Cours\INSA\Git\demo> git branch
* main
  two-images
```

- Let's move to the new branch

```
git checkout two-images
```

```
PS D:\seb\Cours\INSA\Git\demo> git checkout two-images
Switched to branch 'two-images'
PS D:\seb\Cours\INSA\Git\demo> git branch
  main
* two-images
```

Now, the `HEAD` is pointing at the `two-images` branch.



Modify the files

- We now modify `exeDemo2.m`

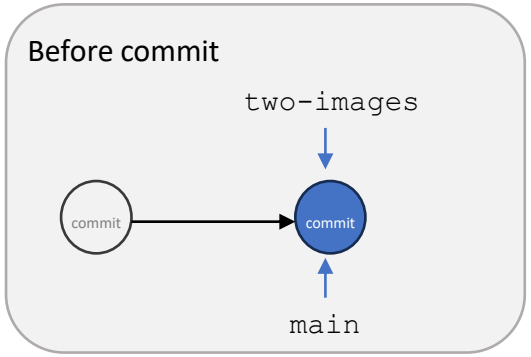
And we add `tortoiseGit.jpg`

```
clearvars
close all
clc

displayImage('gitLogo.png')
% Modification
displayImage('tortoiseGit.jpg')
```

- We can see that Git tracked all the modifications:

```
git status
```

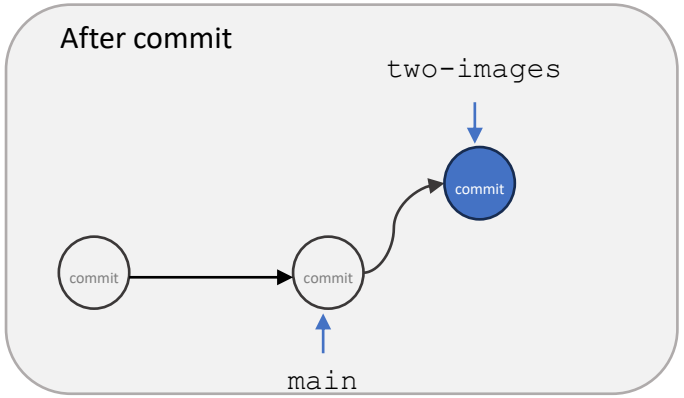
```
PS D:\seb\Cours\INSA\Git\demo> git status
On branch two-images
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   exeDemo2.m

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        tortoiseGit.jpg
```



- Let's add the new file and commit

```
git add -all
git commit -m "Added a second image"
```

The two-image pointer has moved to the new commit and HEAD is still pointing on two-images. (NB: the HEAD is depicted in blue)



More branches

- Let's move to the `main` branch

```
git branch main
```

- Create a new branch for a "quick fix"

```
git branch quick-fix
```

```
PS D:\seb\Cours\INSA\Git\demo> git branch quick-fix
PS D:\seb\Cours\INSA\Git\demo> git branch
* main
  quick-fix
  two-images
```

- Move to this branch

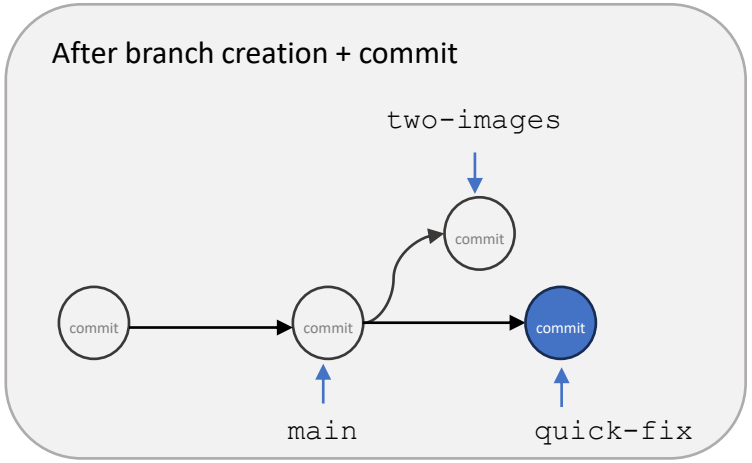
```
git checkout quick-fix
```

- Modify `exeDemo2.m`

```
clearvars
close all
clc

% This is a quick fix : add ";" at the end of the line
displayImage('gitLogo.png');
```

- Now we stage the modified file and commit

```
git add exeDemo2.m
git commit -m "quick fix : missing ;"
```



Merge branches

When merging 2 branches, Git will automatically merge all the branches history.

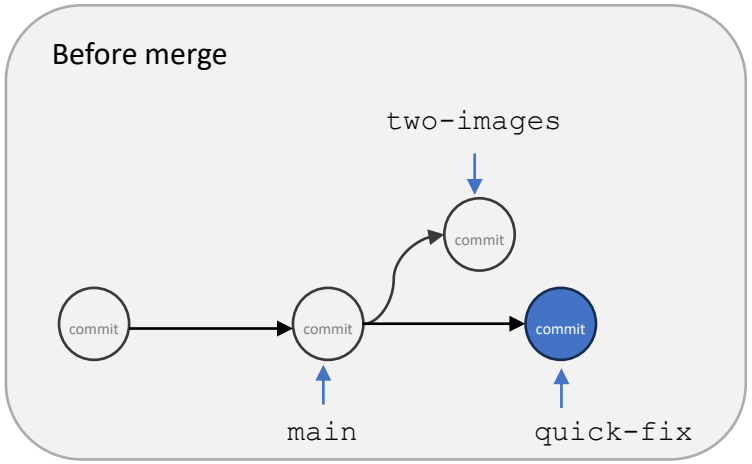
Now we have a branch with a working code. We need to bring the changes to the `main` branch.

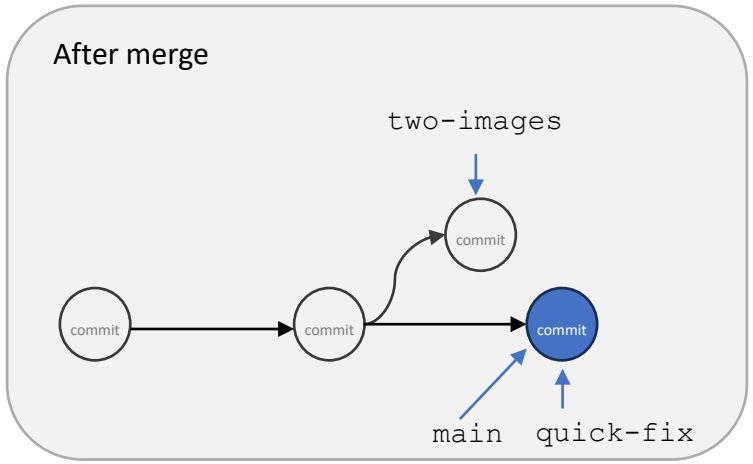
1) Go to the main branch
2) Merge with quick-fix

```
git checkout main
git merge quick-fix
```

```
PS D:\seb\Cours\INSA\Git\demo> git merge quick-fix
Updating 25230db..97299d2
Fast-forward
 exeDemo2.m | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

The result is a "fast-forward" merge : it is sufficient to move the `main` branch to the last commit of `quick-fix`





Merge branches

When merging 2 branches, Git will automatically merge all the branches history.

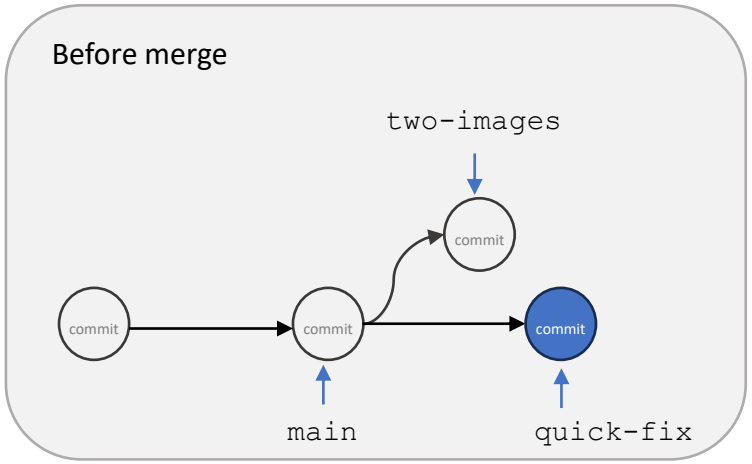
Now we have a branch with a working code. We need to bring the changes to the `main` branch.

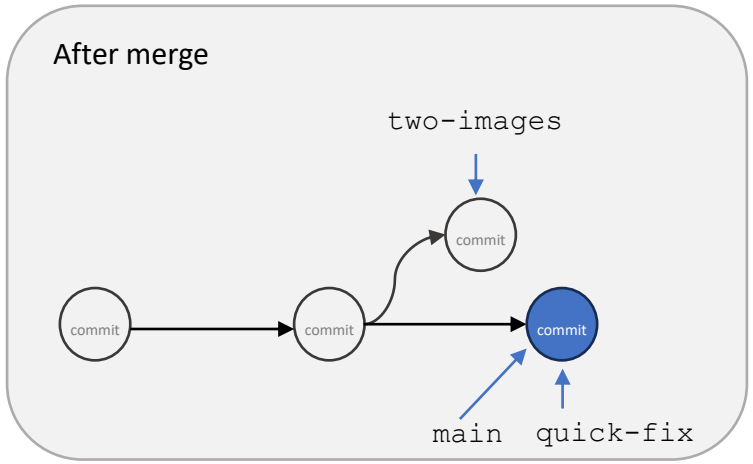
1) Go to the main branch
2) Merge with quick-fix

```
git checkout main
git merge quick-fix
```

```
PS D:\seb\Cours\INSA\Git\demo> git merge quick-fix
Updating 25230db..97299d2
Fast-forward
 exeDemo2.m | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

The result is a "fast-forward" merge : it is sufficient to move the `main` branch to the last commit of `quick-fix`



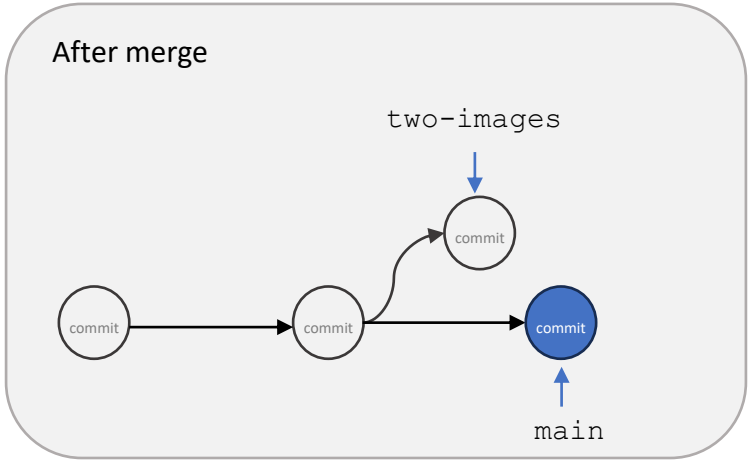


Delete branches

We can now delete the `quick-fix` branch.

```
git branch -d quick-fix
```

```
PS D:\seb\Cours\INSA\Git\demo> git branch -d quick-fix
Deleted branch quick-fix (was 97299d2).
```

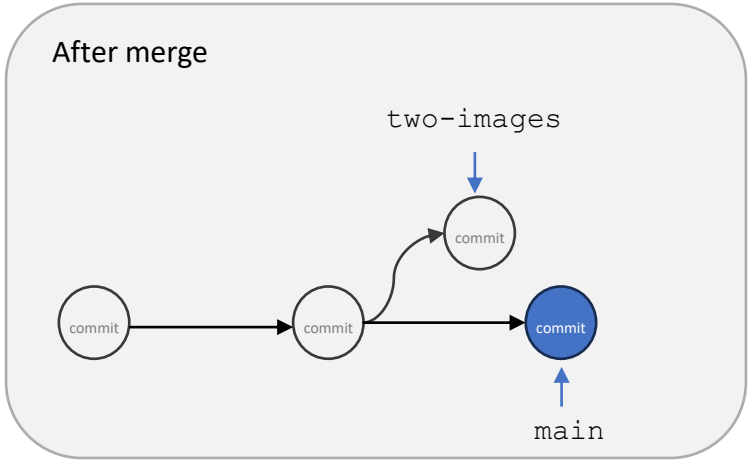


Merge conflict

- What if we try to merge two-images into main?

```
git merge two-images
```

```
PS D:\seb\Cours\INSA\Git\demo> git merge two-images
Auto-merging exeDemo2.m
CONFLICT (content): Merge conflict in exeDemo2.m
Automatic merge failed; fix conflicts and then commit the result.
```



=> A conflict occurs as the `exeDemo2.m` file has been modified in both branches.

```
git status
```

```
PS D:\seb\Cours\INSA\Git\demo> git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
        new file:   tortoiseGit.jpg

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   exeDemo2.m
```

The status command indicated that there is no problem to add the new image "`tortoiseGif.jpg`" but the `exeDemo2.m` causes an issue.

Resolve conflict

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- In order to resolve, the conflict, we need to edit the file `exeDemo2.m`

```
clearvars
close all
clc


<<<<<<< HEAD
% This is a quick fix : add ";" at the end of the line
displayImage('gitLogo.png');
=======


displayImage('gitLogo.png')
% Modification
displayImage('tortoiseGit.jpg')
>>>>>>> two-images
```

Content of the `exeDemo2.m` in the `HEAD` (`main` branch)

Content of the `exeDemo2.m` in the `two-images` branch

Resolve conflict

- We modify the file to fix the conflict in `exeDemo2.m`

```
clearvars
close all
clc


% This is a quick fix : add ";" at the end of the line
displayImage('gitLogo.png');
% Modification
displayImage('tortoiseGit.jpg')
```

- We now stage the modified `exeDemo2.m`

```
git add exeDemo2.m
```

```
git status
```

```
PS D:\seb\Cours\INSA\Git\demo> git add exeDemo2.m
PS D:\seb\Cours\INSA\Git\demo> git status
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   exeDemo2.m
        new file:   tortoiseGit.jpg
```

We see that the conflict is solved but we also need to finish the merge by committing the changes.

- Final commit

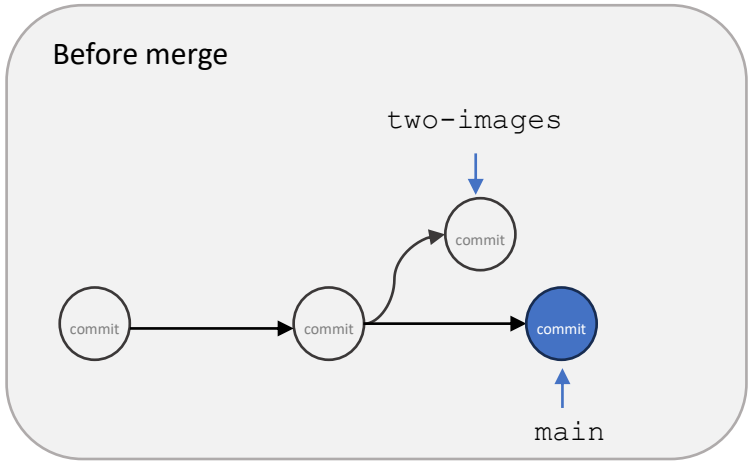
```
git commit -m "merged with two-images after fixing conflicts"
```

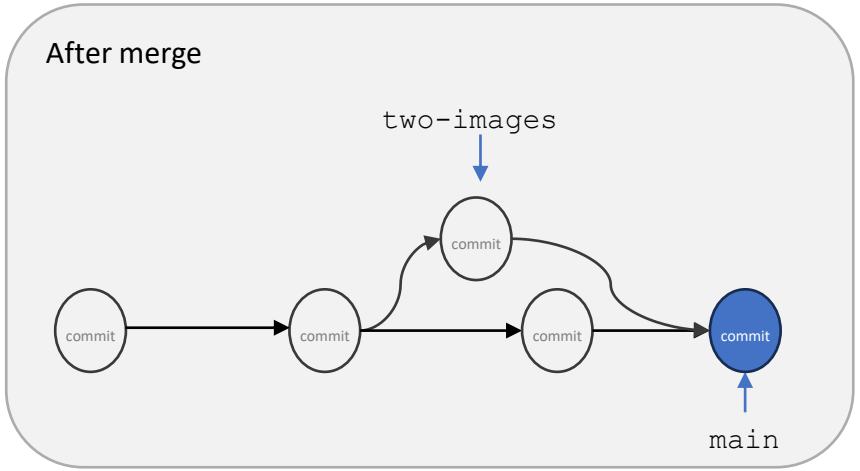
```
PS D:\seb\Cours\INSA\Git\demo> git commit -m "merged with two-images after fixing conflicts"
[main 23009ea] merged with two-images after fixing conflicts
```

Resolve conflict

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Final result:





Eventually, we could delete the `two-images` branch

```
git branch -d two-images
```

GitLab

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

There exist different online repositories

- GitHub
- Gitlab.uphf.fr
- Bitbucket

They allow not only to store code backup but also allow many people to collaborate on the same code.

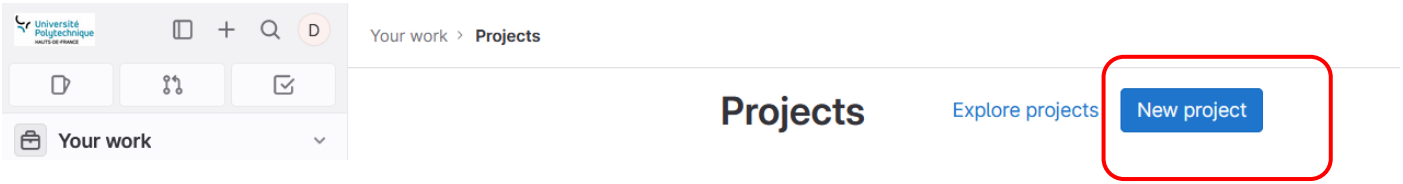
Here we focus on the gitlab server hosted at UPHF.

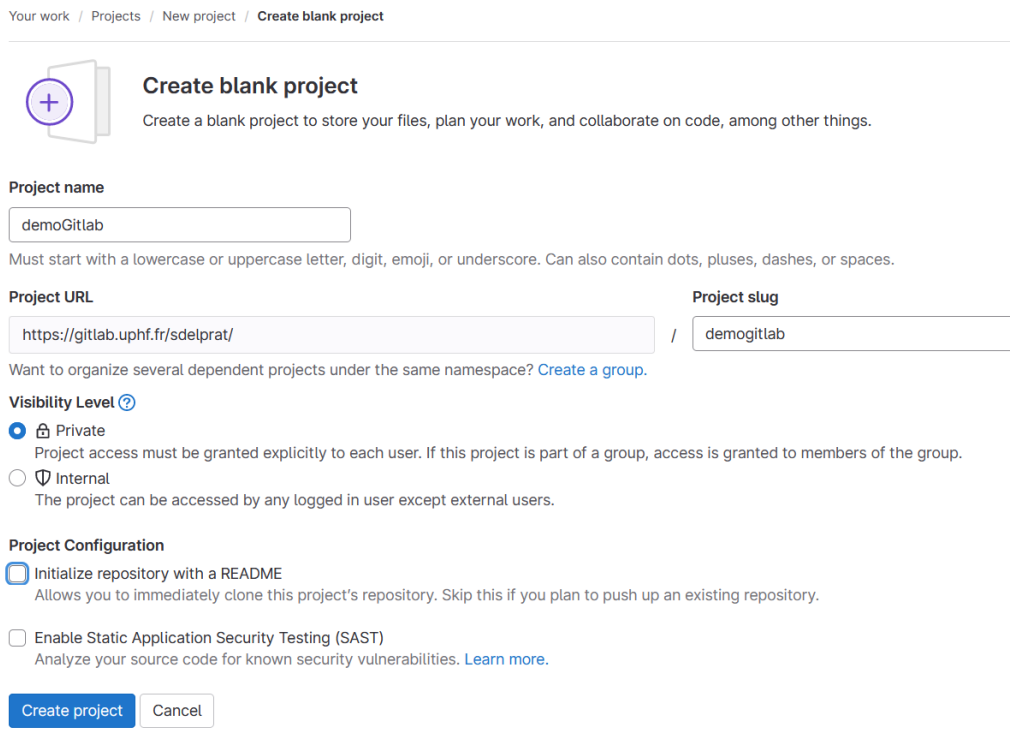
NB : you may use also use github

Create an online repository

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

First, login to `gitlab.uphf.fr` let's create a blank project





Working with SSH

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

SSH is a secure protocol that allows accessing your online repository. It uses a private & public keys.

The procedure is described here :

- GITHUB : https://docs.github.com/fr/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent
- GITLAB : https://docs.gitlab.com/ee/user/ssh.html

Step 1: Generate key using ED25519 cryptographic algorithm

```
ssh-keygen -t ed25519 -C "<comment>"
```

You may use an email adress as comment

```
ssh-keygen -t ed25519 -C "sebastien.delprat@uphf.fr"
```

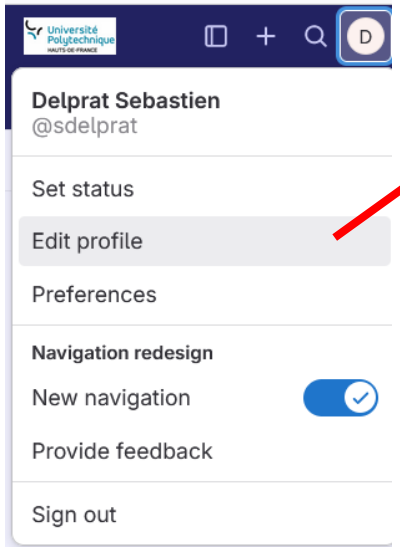
Step 2: The ssh-keygen will ask for a passphrase and will generate the key in default folder

On windows, the default folder is `c:\users\username\.ssh`

You may update the passphrase using the following command

```
ssh-keygen -p -f /path/to/ssh_key
```

Working with SSH



Then copy the content of the .pub file in the .ssh folder

Step 4: Check that connection is working

```
ssh -T git@gitlab.example.com
```

Replace URL with your gitlab hostname

Accept the server fingerprint + type pass phrase when requested

Create an online repository

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Now we would like to copy (push) all the files from our local repository to the server

We first start by indicating where is the remote URL of our server

```
git remote add origin https://gitlab.uphf.fr/sdelprat/demogitlab.git
```

Chosen ID used for the remote server

So now, Git is aware that `origin` maps to the specified URL. We may defined as many remote as we want

NB : Git offers possibility to work with multiples remotes. We can list them, remove some, etc.

```
git remote rm remoteName
```

Remove the specified remote from the list

NB : Git may also ask for some password…

Create an online repository

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

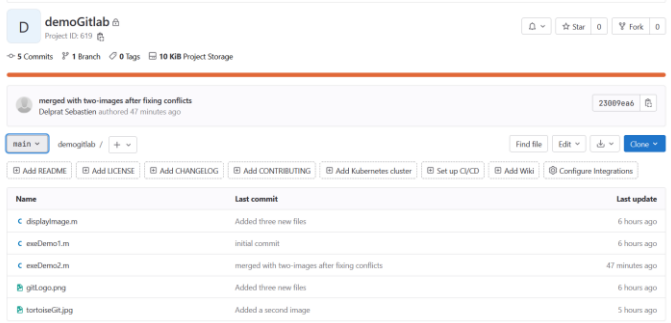
We now indicate we would like to push our `main` branch to the server (`origin`)

full
```
git push --set-upstream origin main
```

short
```
git push -u origin main
```

By setting the `origin` url as upstream of the local `main` we will be able to easily push and pull the files from/to the server

```
PS D:\seb\Cours\INSA\Git\demo> git push --set-upstream origin main
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 12 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (18/18), 12.78 KiB | 6.39 MiB/s, done.
Total 18 (delta 6), reused 0 (delta 0), pack-reused 0
To https://gitlab.uphf.fr/sdelprat/demogitlab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```



Copy an online repo to a local folder

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Once the files are on the server, it becomes possible to clone the repository to a local folder.

- Navigate to the folder where you would like to download the repo
- Clone the repo with the following command

```
git clone url folderName
```

```
PS D:\seb\Cours\INSA\Git> git clone https://gitlab.uphf.fr/sdelprat/demogitlab.git demo2
Cloning into 'demo2'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.

Receiving objects: 100% (18/18), 12.78 KiB | 12.78 MiB/s, done.
Resolving deltas: 100% (6/6), done.
```

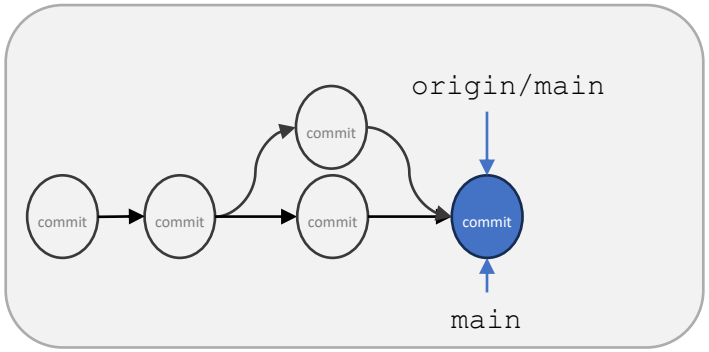
Local & remote branches

We have local and remotes branches. We can also list all the branches with the `-a` option.

```
git branch -a
```

```
PS D:\seb\Cours\INSA\Git\demo> git branch -a
* main
  remotes/origin/main
```

Now we need to distinguish the local `main` from the remote `origin/main`



When many people collaborate together, they can make change to the `origin/main` branch. So this branch may evolve independently of the local user!!

Pushing modification to the server

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

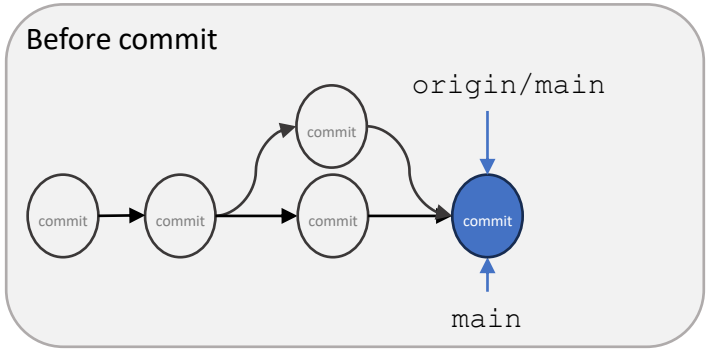
- Let us modify some files

`exeDemo1.m`

```
clearvars
close all
clc

% This is a first file into the repository
fprintf('My first file\n');
fprintf('It is now on the server\n');
```

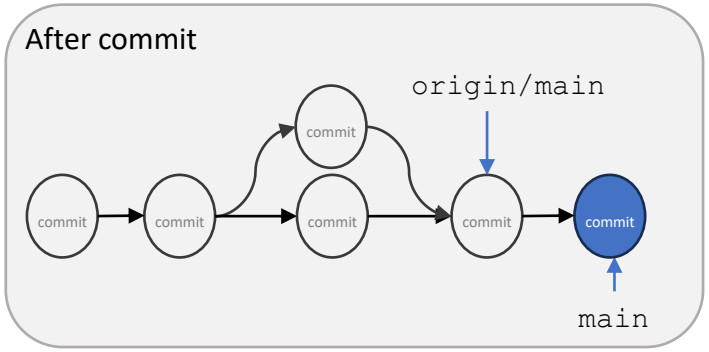
`exeDemo3.m`

```
clearvars
close all
clc


figure;
plot(rand(2,20))
```



- We can now stage files & commit

```
git add .
git commit -m "local modification of some files"
```



- Git is aware that our local modifications are not yet on the server

```
git status
```

```
PS D:\seb\Cours\INSA\Git\demo> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

It is clear that the local `main` branch is ahead of the `origin/main` one.

Pushing modif

WRONG IMAGES

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

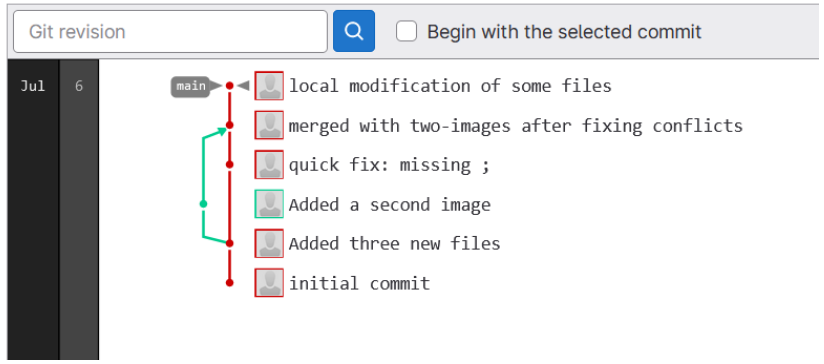
- We now push the files

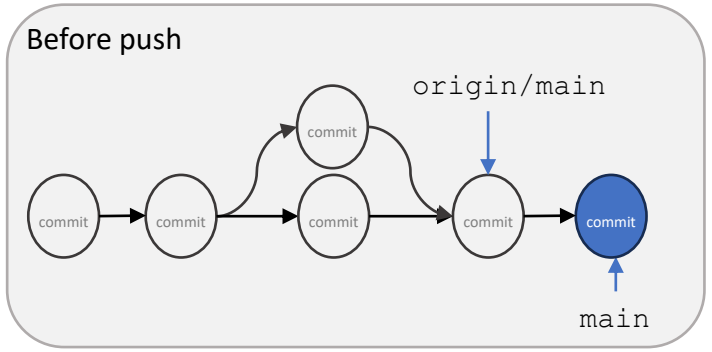
```
git push origin
```

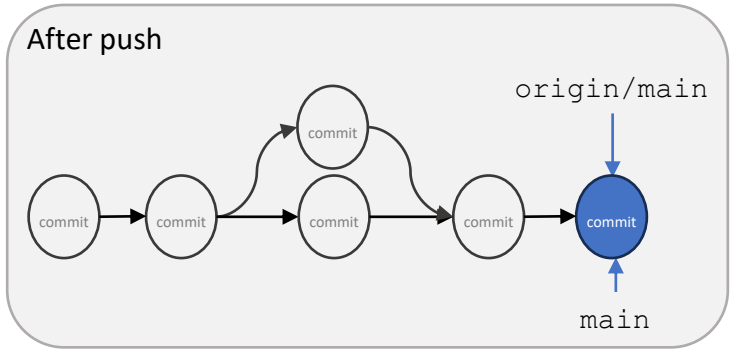
Origin refers to the URL set by `git remote add origin URL`

```
PS D:\seb\Cours\INSA\Git\demo> git push origin
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 436 bytes | 436.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
To https://gitlab.uphf.fr/sdelprat/demogitlab.git
   23009ea..2bc1e5d  main -> main
```

Menu Code/Repository graph





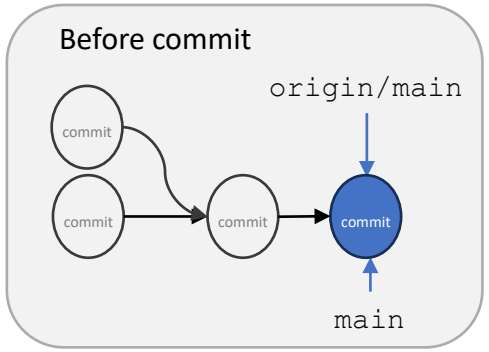


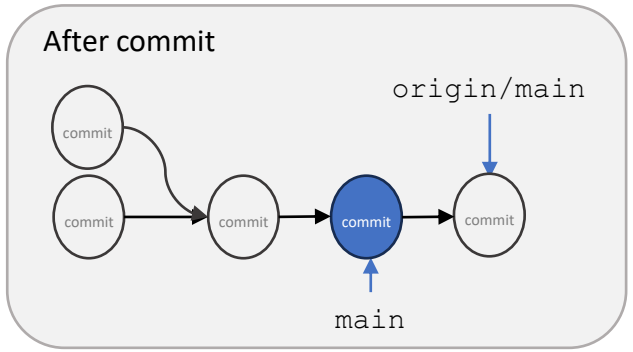
Modifying files on the server

- Files can be modified by direct edition on the server using the web interface, or modified by another project member (using a `push` command)

Let us add a line to `exeDemo1.m` and commit changes using the interface

```
clearvars
close all
Clc

% This is a first file into the repository
fprintf('My first file\n');
fprintf('It is now on the server\n');
fprintf('This is a modification done on the server (origin)\n');
```





- We can now check the status of our local repository:

```
git status
```

```
PS D:\seb\Cours\INSA\Git\demo> git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean
```

Our local code is not up to date, we need to pull the server files

Finding differences

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- Git allows to find differences between the current branch and another one

```
git diff origin/main
```

```
PS D:\seb\Cours\INSA\Git\demo> git diff origin/main
diff --git a/exeDemo1.m b/exeDemo1.m
index dccce1c..809a3c4 100644
--- a/exeDemo1.m
+++ b/exeDemo1.m
@@ -5,6 +5,4 @@ clc

 % This is a first file into the repository
 fprintf('My first file\n');
-fprintf('It is now on the server\n');
-
-fprintf('This is a modification done on the server (origin)\n');
+fprintf('It is now on the server\n');
\ No newline at end of file
```

Obviously using a GUI such as source Tree allows better visualization

exeDemo1.m

Bloc 1 : lignes 5-10 — Restaurer le fragment

```
5   5   ...
6   6   ...% This is a first file into the repository
7   7   ...fprintf('My first file\n');
8       -..fprintf('It is now on the server\n');
        \..No newline at end of file
    8   +.fprintf('It is now on the server\n');
    9   +.
    10  +.fprintf('This is a modification done on the server (origin)\n');
```

Fetch files from the server

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

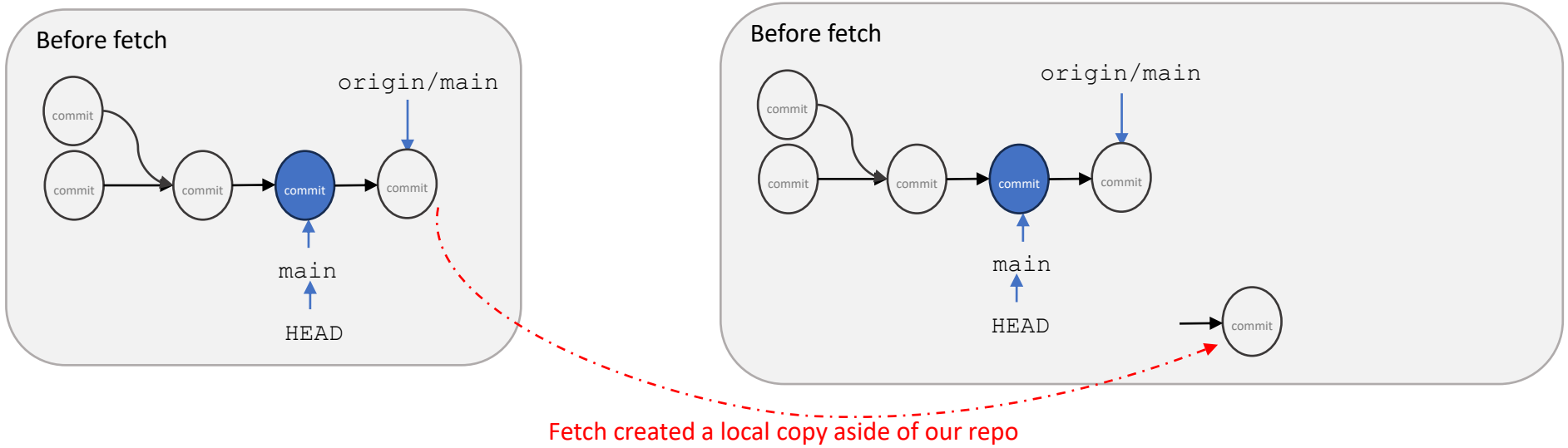
- Files can be modified by direct edition on the server, or modified by another project member
- We can download the `origin/main` branch content using fetch

```
git fetch origin main
```

Reference to remote repository

Name of the remote branch

```
PS D:\seb\Cours\INSA\Git\demo> git fetch origin main
From https://gitlab.uphf.fr/sdelprat/demogitlab
 * branch            main       -> FETCH_HEAD
```



- The outcome is a local copy of the commit from the remote server but detached from our branch

Fetch files from the server

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

- As we now we have a local copy of the origin/main branch, we can check it out

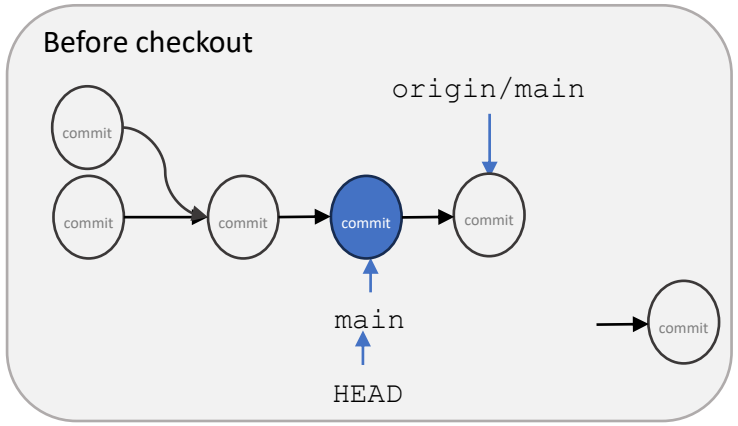
```
git checkout origin/main
```

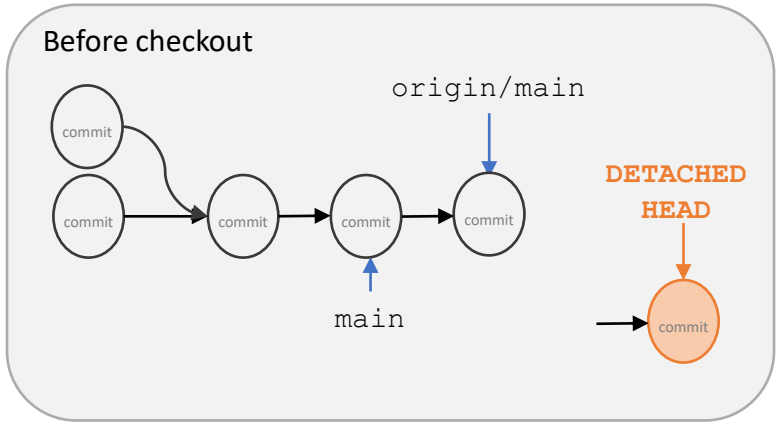
- The local copy of the `origin/main` being separated from our local main branch, the checkout will not use a regular `HEAD` but a `DETACHED HEAD`.

```
git status
```

```
PS D:\seb\Cours\INSA\Git\demo> git status
HEAD detached at origin/main
nothing to commit, working tree clean
```

```
PS D:\seb\Cours\INSA\Git\demo> git checkout origin/main
Note: switching to 'origin/main'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 20cd4b7 Update exeDemo1.m
```





Merging files from the server

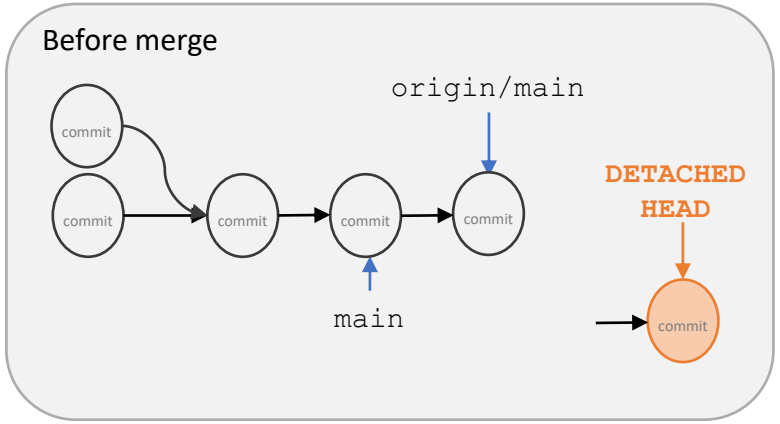
INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

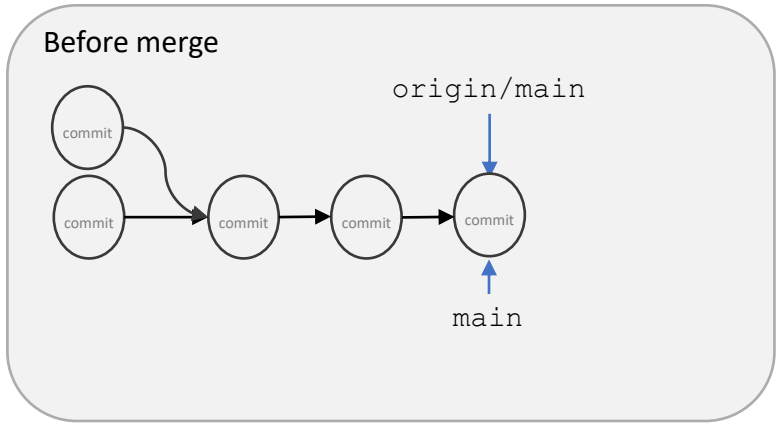
- We can now look at the files, eventually modify them
- We can create a new branch with this data into our own history using the `branch` command

- We now decide to merge this branch with our local `main`

```
git checkout main
git merge origin/main
```

```
PS D:\seb\Cours\INSA\Git\demo> git merge origin/main
Updating 2bc1e5d..20cd4b7
Fast-forward
 exeDemo1.m | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)
```





Pulling files from the server

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

In order to update our local history with the data from the server, we used 2 commands:

```
git fetch origin main
git merge origin/main
```

It is possible to do both operations at the same time using the `pull` command

```
git pull origin
```

The `pull` command just execute a `fetch` command of the current branch on the specified remote followed by a `merge`.

```
PS D:\seb\Cours\INSA\Git\demo> git pull origin
Updating 2bc1e5d..20cd4b7
Fast-forward
 exeDemo1.m | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)
```

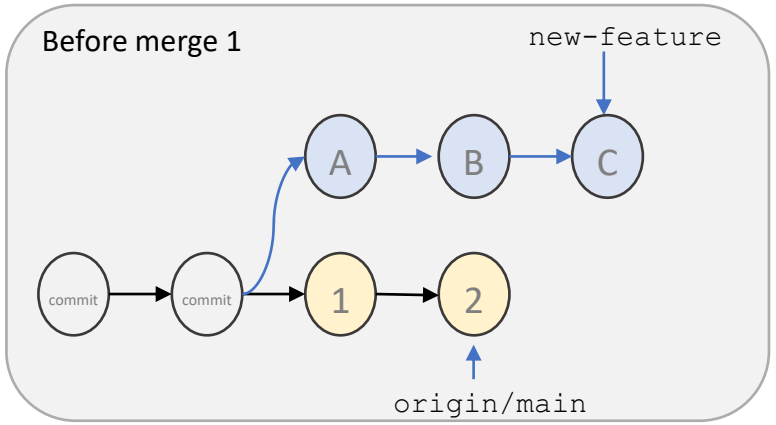
Merge vs Rebase

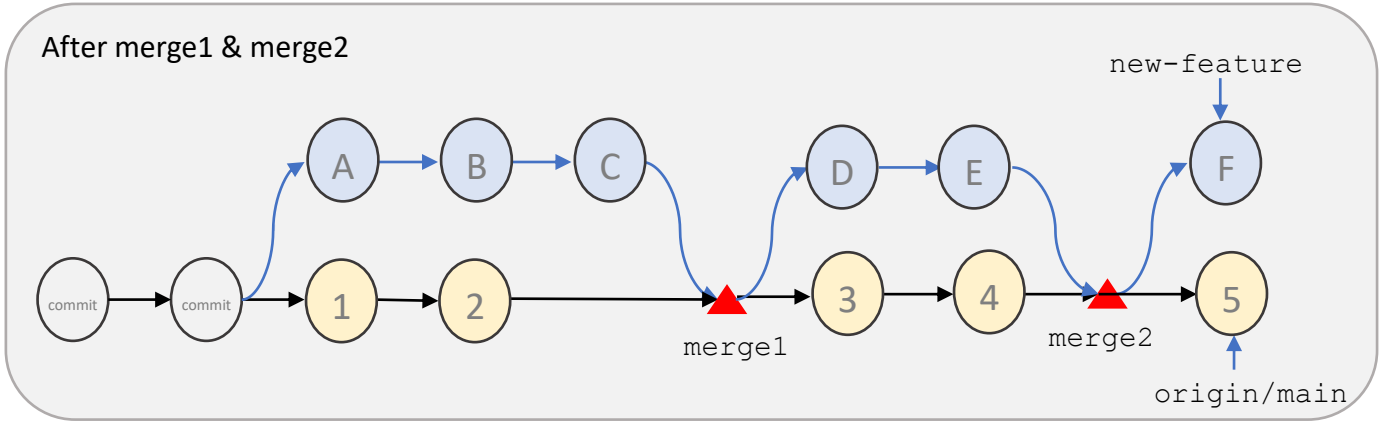
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Let us imagine that we are working on a `new-feature` branch of our code while other colleagues are developing their own features.

While you develop `new-feature`, the `origin/main` has evolved.

You need to keep up to date with the `origin/main` while working on the `new-feature` branch. We need to periodically do sor `merge`





The workflow is working, but the history contains numerous merges.
The history is indeed complete

Merge vs Rebase

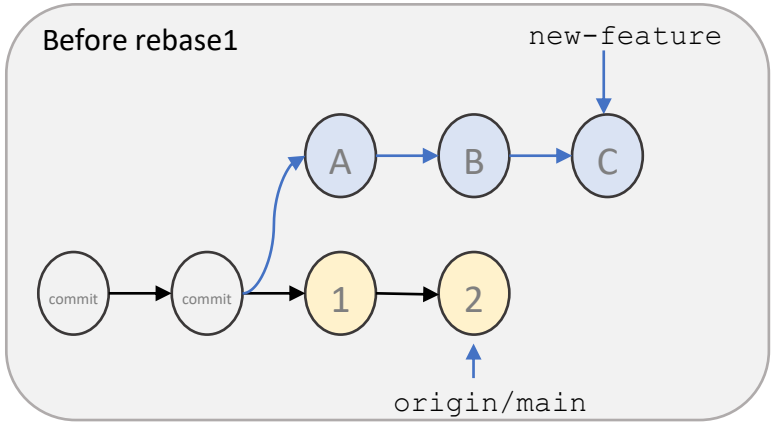
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

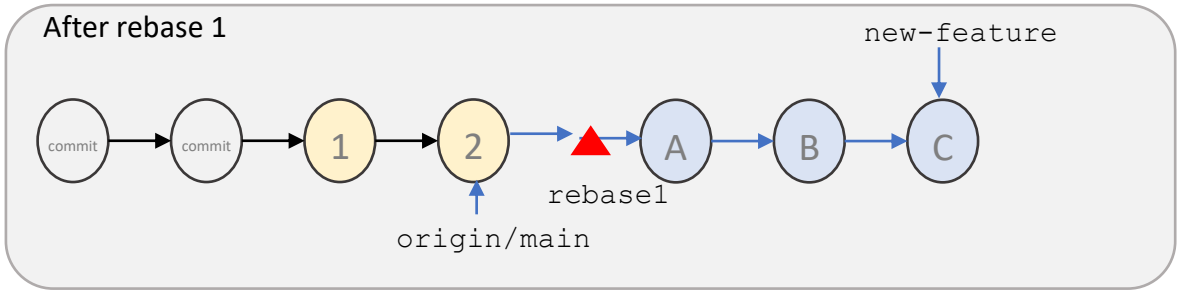
Now let's do the same thing with the rebase option.

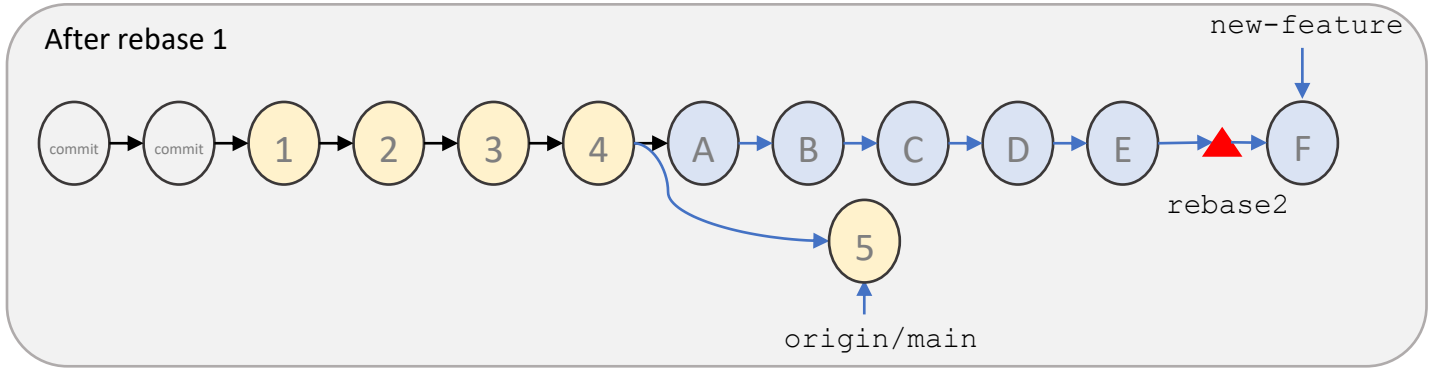
```
git fetch origin
git rebase origin/mail
```

```
git pull origin --rebase
```

It will reorganize the commits in such a way that they are always ahead of the `origin/main`.







=> The commits of the new-feature branch are always located after

Merge vs Rebase

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

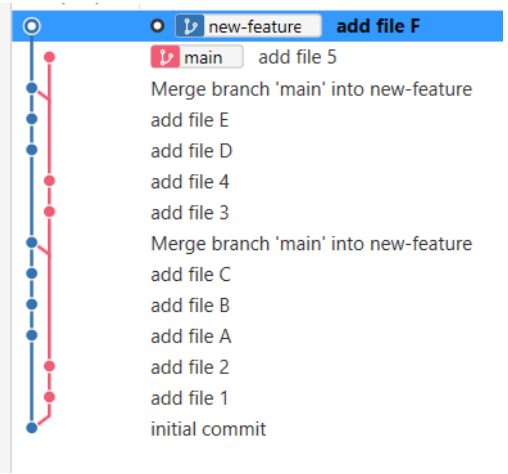
The `merge` operation is consistent and reflect exactly what happened, with a preserved chronological order. The readability of the history can be limited, especially when working with multiple branches

The `rebase` operation is not time consistent but lead to simpler history.

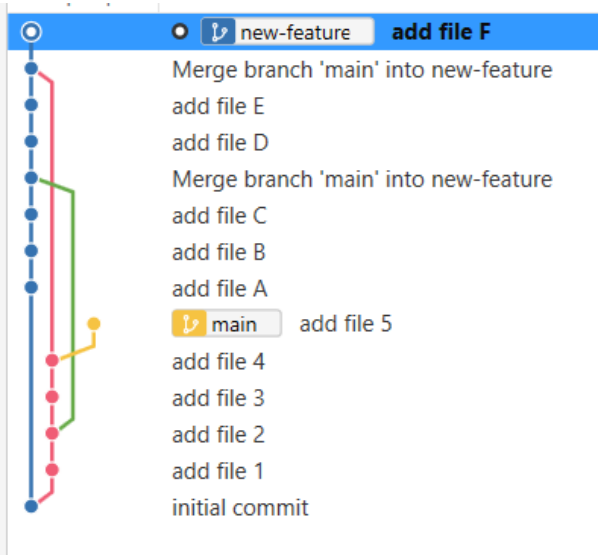
These are the results of exactly the same git command except `merge`/`rebase`

Using `merge`

This is a chronological view

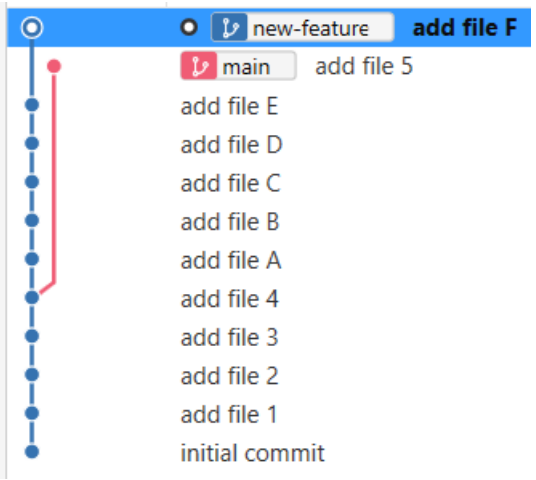


This is an ancestor view

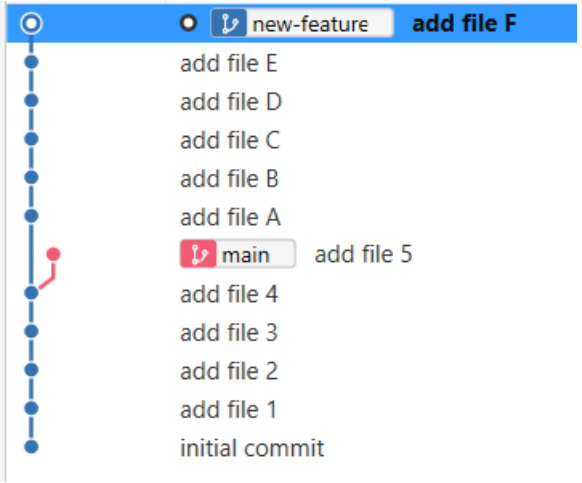


Using `rebase`

This is a chronological view



This is an ancestor view



Workflow, branches

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

In general, it is not advised to let user pushing their branch to the `origin/main`.

Gitlab, Github, etc have an interface that allows setting permissions to users.

Every project may have different rules. Let's investigate a basic workflow

- Main branch always contains validated code
- Each feature is developed on its own branch
- Developers work on their local copy of the branch and push their commit on a regular basis
- Once a feature is advanced enough, a pull/merge request is send
- After every request, the code is reviewed by the project member, eventually modified
- Finally, the request is accepted and the merge is done.

Nb: Github uses "pull request" whereas Gitlab uses "merge request"

Workflow, branches

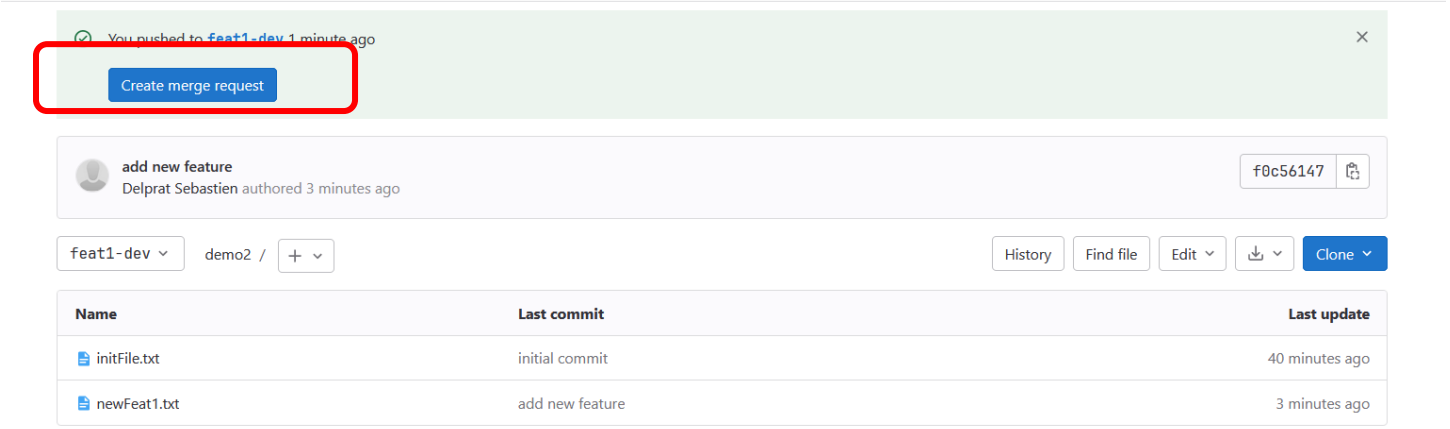
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

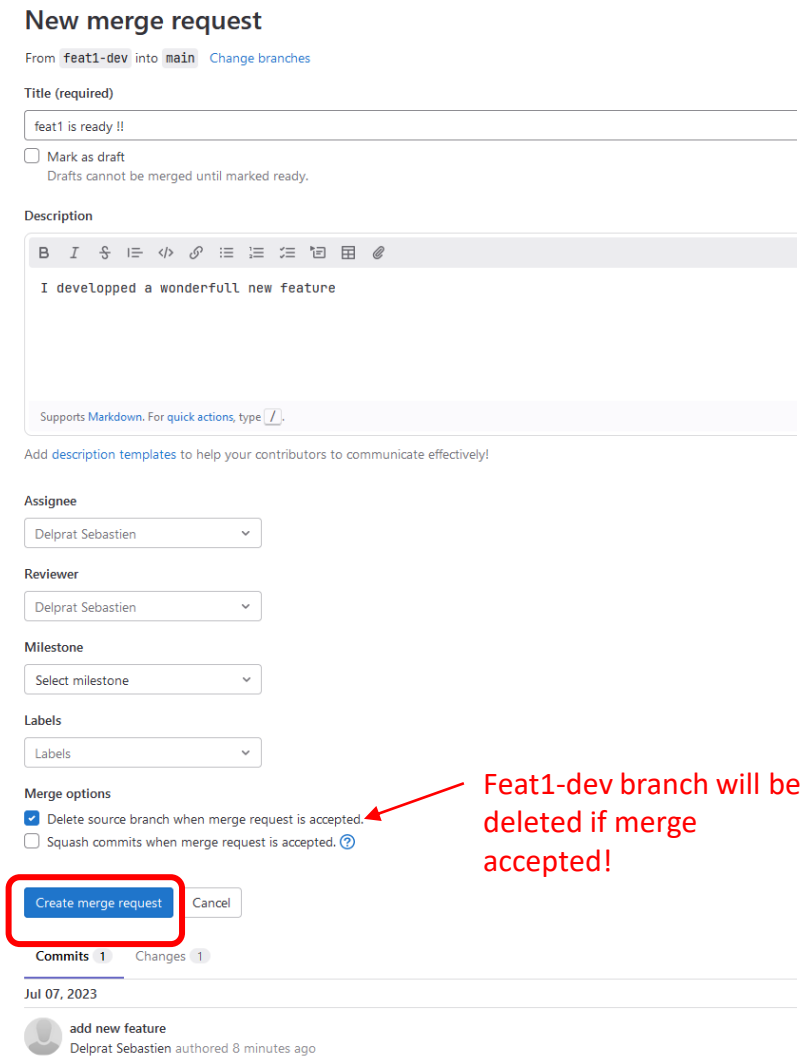
So let's start a new project, with some commit on the `origin/main` branch
Add also add a `feat1-dev` branch

```
git pull
git checkout feat1-dev
new-item newFeat1.txt
git add .
git commit -m "add new feature"

git push origin feat1-dev
```

Windows PowerShell command to create a file

Now our `feat1-dev` is ready, we create a merge request





Feat1-dev branch will be deleted if merge accepted!

Workflow, branches

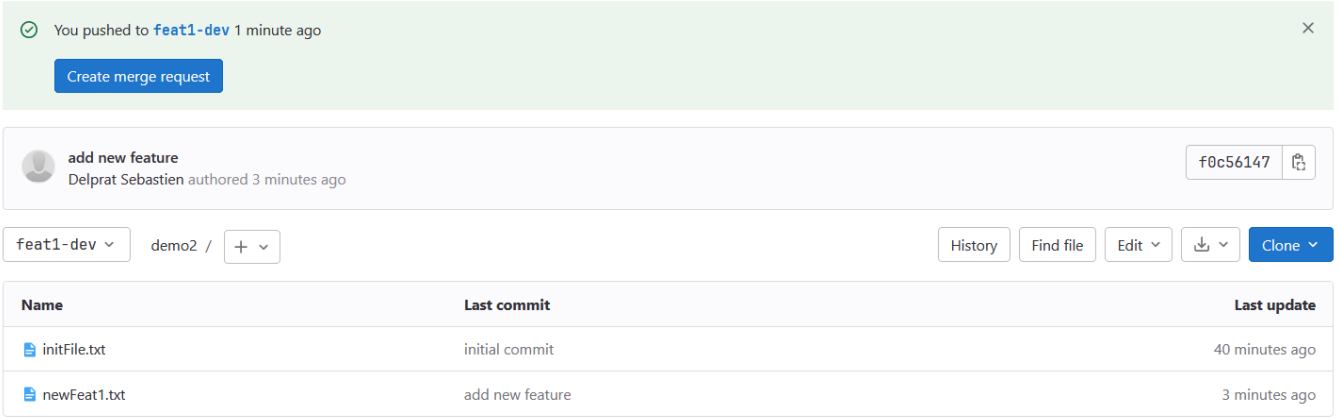
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

So let's start a new project, with some commit on the `origin/main` branch
Add also add a `feat1-dev` branch

```
git pull
git checkout feat1-dev
new-item newFeat1.txt
git add .
git commit -m "add new feature"

git push origin feat1-dev
```

Windows PowerShell command to create a file

Now our `feat1-dev` is ready, we create a merge request

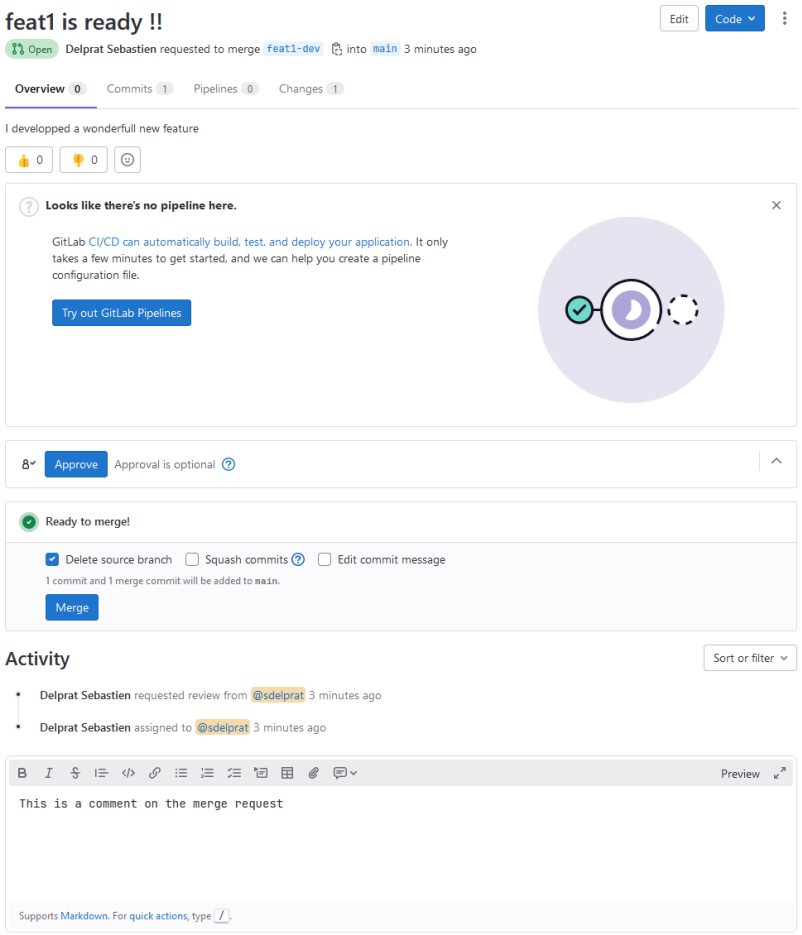


Workflow, branches

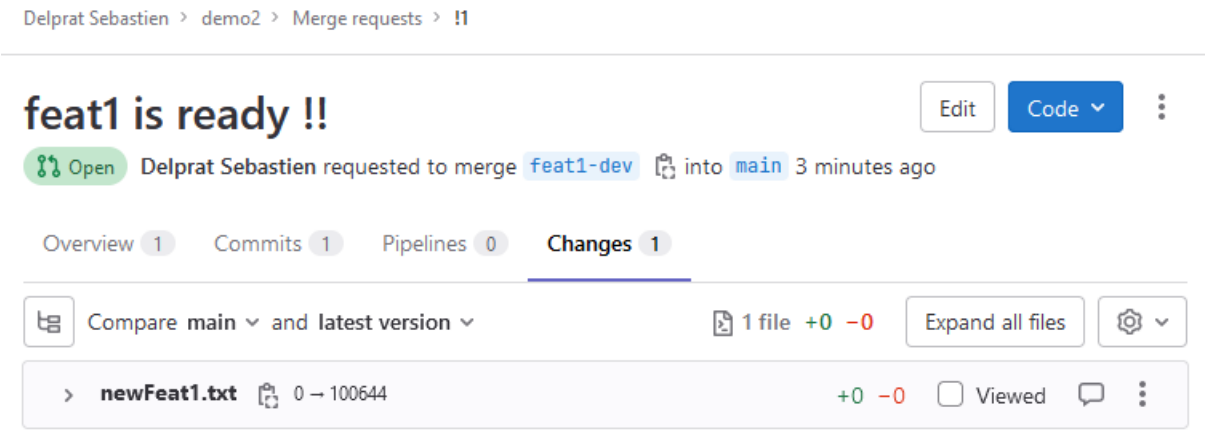
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

The reviewers receive the merge request in their interface

Code can be compared and commented



Changes can be viewed online



Workflow, branches

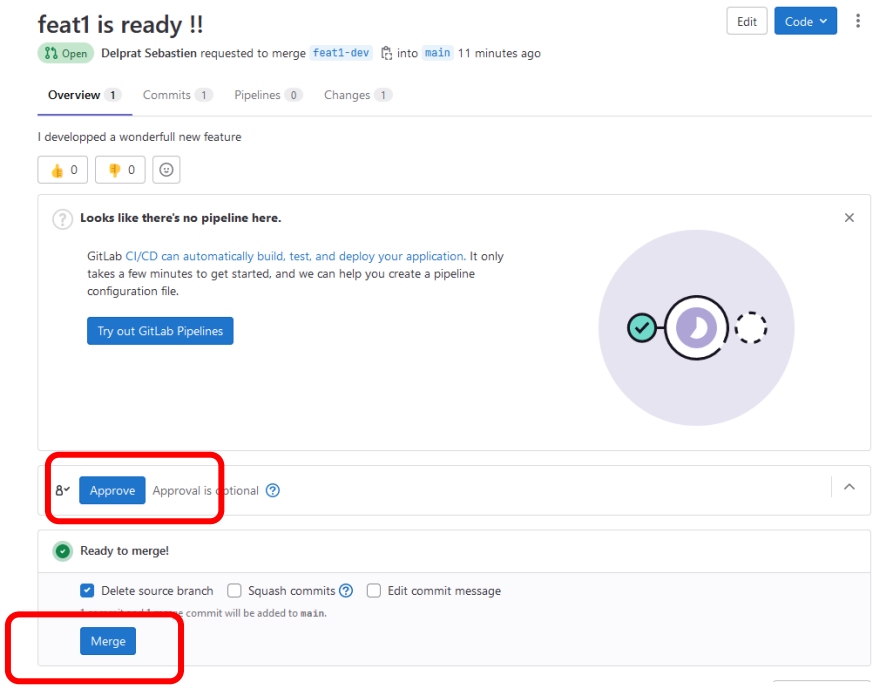
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

On real project with large development teams, several review steps may be needed before the merge request is approved

An example of a complete workflow is described in the following video:

https://www.youtube.com/watch?v=muozG1gyOqg

So in the end, the merge request is approved and the merge is performed



Revert back

INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

One of the main advantage of keeping the history of modification is that you can revert back the code to a given state.

Each commit is associated with an hexadecimal reference

```
git log --oneline
```

```
23009ea (HEAD -> main) merged with two-images after fixing conflicts
97299d2 quick fix: missing ;
e567638 Added a second image
25230db Added three new files
4166dc1 initial commit
```